

Club de Algoritmos UAM Iztapalapa, Licenciatura en
Computación.

NOTAS PARA UNA INTRODUCCIÓN A LA PROGRAMACIÓN COMPETITIVA EN C++

Ing. Luis Daniel Ramos López

Actualizadas:

17/04/2026

Índice general

Cabeceras e inicio de programa	3
Biblioteca <bits/stdc++.h>	3
Uso de using namespace std	3
Desincronización de iostream con stdio.h	3
Compilación y ejecución	4
Algunas bases de programación estructurada	4
Leer y escribir variables	4
Estructuras de condición y repetición	4
Declaración de arreglos	6
Tipos de datos	6
Algoritmia y eficiencia	7
Límite en tiempo	7
Límite en memoria	7
Problemas clásicos de algoritmia sobre arreglos	8
Ocorrencias en una lista	8
Sumas de prefijos en una lista	9
Mayor suma de un subarreglo de tamaño k (ventana deslizante)	11
Apuntadores, la biblioteca algorithm y estructuras	12
Apuntadores	12
Algunas funciones básicas de la biblioteca algorithm	13
Declaración de estructuras	13
Algoritmos de ordenamiento y de búsqueda de secuencias	14
Ordenamiento de secuencias	14
Búsqueda eficiente sobre secuencias ordenadas	15
Estructuras de datos lineales	16
Cadenas de caracteres (string)	16
Arreglos dinámicos (vector)	17
Pilas (stack)	18
Colas (queue)	18
Dobles colas (deque)	19
Listas enlazadas (list)	20
Estructuras de datos no lineales	21
Conjuntos (set)	21
Diccionarios (map)	22
Colas de prioridad (priority_queue)	23
Algunos algoritmos art	24
Operaciones con módulo	24
Criba de Eratóstenes	25
Coeficiente binomial iterativo	25
Potencia rápida con módulo	26
Binomial iterativo con módulo	26
Factorización básica	27
Máximo común divisor y mínimo común múltiplo	28
Algunas consideraciones adicionales	28
Uso de auto	28
¿Qué hacer cuando voy a resolver un problema?	29

Cabeceras e inicio del programa

Biblioteca <bits/stdc++.h>

Para evitar poner cada biblioteca necesaria que se va a ocupar en un programa, se puede utilizar una biblioteca de C++ que **solo funciona con el compilador GCC** que permite utilizar cualquier función u objeto de la biblioteca estándar de C++ sin tener que ponerlas por separado.

```
#include <bits/stdc++.h>
```

Sin bits/stdc++.h:

```
1. #include <iostream>
2. #include <string>
3.
4. int main() {
5.     std::string s;    //Para usar cadenas usamos la biblioteca string
6.     std::cin >> s;    //Para entrada y salida estándar usamos la biblioteca iostream
7.     std::cout << s;
8. }
```

Con bits/stdc++.h:

```
1. #include <bits/stdc++.h>
2.
3. int main() {
4.     std::string s;
5.     std::cin >> s;    //Aquí solo usamos un "include" en vez de dos
6.     std::cout << s;
7. }
```

Uso de using namespace std

Para evitar usar el prefijo “std:.” para cada objeto o función de la biblioteca estándar de C++, usamos la directiva **using namespace std**.

```
1. #include <iostream>
2.
3. int main() {
4.     std::cout << "Hola mundo";
5. }
```

```
1. #include <iostream>
2. using namespace std;
3.
4. int main() {
5.     cout << "Hola mundo";
6. }
```

Desincronización de iostream con stdio.h

Para evitar la sincronización de iostream con stdio.h, lo cual es importante hacer cuando hay muchos elementos que leer o escribir **solo con iostream**, se usan dos líneas:

```
1. int main() {
2.     ios_base::sync_with_stdio(false);
3.     cin.tie(nullptr), cout.tie(nullptr);
4.     //... lo que sigue de código
5. }
```

Compilación y ejecución

Para compilar un archivo con extensión .cpp en la terminal, debemos estar posicionados en la carpeta que contiene el archivo y compilarlo de la siguiente manera:

```
g++ archivo.cpp -o archivo
```

Es importante saber que en un ambiente Linux los ejecutables, en general, no tienen extensión, a diferencia del .exe de los ejecutables en Windows. Para ejecutar en un ambiente de Linux, lo hacemos de la siguiente manera:

```
./archivo
```

Algunas bases de programación estructurada

Leer y escribir variables

Para leer y escribir variables desde la entrada estándar de cualquier tipo se utiliza **cin** (para leer) y **cout** (para escribir) de la biblioteca **iostream**:

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int main() {
5.     int n;
6.     char c;
7.     string s;
8.     float f;
9.     cin >> n >> c >> s >> f;
10.    cout << "entero: " << n << "   flotante: " << f << "\n";
11.    cout << c << " " << s << "\n";
12. }
```

Entrada:

```
5
p
gatito
3.14
```

Salida:

```
entero: 5   flotante: 3.14
p gatito
```

Nota: Evitar utilizar endl para salto de línea, en su lugar usen “\n”.

Estructuras de condición y repetición

Se pueden usar una condición **if** solamente:

```
if(condición) {
    //código
}
```

También la puedes concatenar con un **else**:

```
if(condición) {  
    //código  
} else {  
    //código  
}
```

Aunque también puedes concatenar varias condicionales:

```
if(condición) {  
    //código  
} else if(condición) {  
    //código  
} ... else {  
    //código  
}
```

También se puede usar la estructura **switch**:

```
switch (var) {  
case 1:  
    //código  
    break;  
case 2:  
    //código  
    break;  
...  
default:  
    //código  
    break;  
}
```

Como estructuras de repetición tenemos la estructura **while**:

```
while(condición) {  
    //código  
}
```

La estructura **do-while**:

```
do {  
    //código  
} while(condición);
```

Y la estructura **for** que, en este ejemplo, imprime los números del 0 al 2:

```
// var.  cond.  inc.  
for(int i = 0; i < 3; ++i) {  
    cout << i << "\n";  
}
```

Salida:

```
0  
1  
2
```

Declaración de arreglos

Para declarar un arreglo nosotros usamos la sintaxis **tipo nombre[tam]**, donde **tipo** es el tipo de dato (u objeto) de nuestros elementos, **nombre** es el nombre de nuestro arreglo y **tam** es un entero que determina el tamaño del arreglo.

```
int arr[5]; //arreglo de cinco enteros
```

También podemos usar una variable de tipo entera para declarar el tamaño del arreglo:

```
int n = 3;  
string nom[n]; //arreglo de cadenas de 3 elementos.
```

Para acceder a un elemento del arreglo, para leerlo, modificarlo o imprimirlo, se usan los corchetes []:

```
cin >> arr[3]; //leo un valor para el elemento 3 del arreglo  
arr[1] = 5; //le asigno un 5 al elemento 1 del arreglo  
cout << arr[i]; // imprimo el i-ésimo elemento del arreglo (podemos declarar a i en un for)
```

Al declarar un arreglo, las direcciones de memoria traen por defecto basura, no es correcto suponer que tendrán cero en sus elementos inicialmente. Para declarar un arreglo de enteros con puros ceros de inicio se hace de este modo:

```
int arr[1000] = {} //arreglo de 1000 elementos declarado inicialmente con ceros
```

Tipos de datos

A continuación, se muestra una tabla con los tamaños de los tipos de datos principales en C++, para saber si es conveniente usar un tipo de dato o puede desbordarse.

Tipo	Descripción	Tamaño	Rango de valores
bool	Valor booleano	1 bytes	<i>true</i> o <i>false</i>
short	Entero corto con signo	2 bytes	-3.2×10^4 a 3.2×10^4
unsigned short	Entero corto sin signo	2 bytes	0 a 6.5×10^4
int	Entero con signo	4 bytes	-2.1×10^9 a 2.1×10^9
unsigned int	Entero sin signo	4 bytes	0 a 4.3×10^9
long long / int64_t	Entero largo con signo	8 bytes	-9.2×10^{18} a 9.2×10^{18}
unsigned long long / uint64_t	Entero largo sin signo	8 bytes	0 a 1.8×10^{19}
float	Punto flotante	4 bytes	$\pm 3.4 \times 10^{38}$ (7 dígitos de precisión)
double	Doble precisión	8 bytes	$\pm 1.7 \times 10^{308}$ (15 dígitos)
long double	Doble precisión extendida	16 bytes*	$\approx \pm 10^{4932}$ (mayor precisión)

*Nota: el tamaño de long double puede variar según el compilador (a veces 12, 16 o incluso 8 bytes).

Algoritmia y eficiencia

Límite en tiempo

Un procesador puede hacer aproximadamente 10^8 instrucciones por segundo, por eso es importante saber la complejidad de los algoritmos que vamos a utilizar, el tiempo que tenemos disponible también depende de cada problema. Como muchas complejidades dependen del tamaño de la entrada, en la siguiente tabla se muestra un pequeño resumen de cuál puede ser la complejidad máxima recomendada con respecto al tamaño de la entrada:

Tamaño de entrada n	Complejidad máxima recomendada	Comentario
10^2	$O(n^4)$	Todo pasa sin problema
10^3	$O(n^3)$	Clásico de fuerza bruta
10^4	$O(n^2)$	Ya empieza a ser el límite
10^5	$O(n \log(n))$	Ordenar, colas de prioridad, etc.
$> 10^6$	$O(n)$	Recorridos simples

Una manera “estándar” de saber la complejidad temporal del algoritmo que estás implementando en tu programa es preguntarte ¿de qué tamaño es la entrada?, para después preguntarte ¿cuántas veces estoy recorriendo lo equivalente al tamaño de la entrada? Si tu entrada fue una lista de n elementos, entonces la cantidad de for anidados de tu algoritmo es equivalente a la complejidad en tiempo, se puede ver fácilmente de la siguiente manera:

```
for(int i = 0; i < n; ++i) {    //Complejidad lineal  $O(n)$ .
    ...
    for(int i = 0; i < n; ++i) {    //Complejidad cuadrática  $O(n^2)$ .
        ...
        for(int i = 0; i < n; ++i) {    //Complejidad cúbica  $O(n^3)$ .
            ...
        }
    }
}
```

Existen complejidades un poco más complicadas, como la exponencial o la logarítmica, pero para usos prácticos de un curso de introducción a la programación competitiva, las polinomiales deberían de ser suficientes.

Límite en memoria

También, es importante el límite de memoria, puesto que declarar muchas variables puede ocupar más memoria de la que tenemos disponible. Debido a esto, se muestra una tabla con el tamaño de un arreglo de 10^6 elementos de los tipos de datos principales en C++.

Tipo	Tamaño por elemento	Memoria de un arreglo de (10^6) elementos
bool	1 byte	≈ 1 MB
short	2 bytes	≈ 2 MB
unsigned short	2 bytes	≈ 2 MB
int	4 bytes	≈ 4 MB
unsigned int	4 bytes	≈ 4 MB

Tipo	Tamaño por elemento	Memoria de un arreglo de (10 ⁶) elementos
long long / int64_t	8 bytes	≈ 8 MB
unsigned long long / uint64_t	8 bytes	≈ 8 MB
float	4 bytes	≈ 4 MB
double	8 bytes	≈ 8 MB
long double	16 bytes*	≈ 16 MB

Para un arreglo de mayor tamaño que 4MB es necesario declararlo de manera global, es decir, fuera del `main`. La memoria disponible se puede consultar en cada problema.

Problemas clásicos de algoritmia sobre arreglos

Ocurrencias en una lista

Te dan una lista de N elementos, y después te dan M preguntas, de saber cuántas veces aparece el entero k en la lista de N elementos.

Entrada

Un entero N , seguido de N enteros que es la lista de elementos. Después un entero M seguido de M enteros k , que son las preguntas.

Salida

Para cada entero k , un entero que sea la cantidad de veces que aparece k en la lista de N elementos.

Ejemplo:

Entrada

```
5
1 3 1 5 3
4
1 2 3 5
```

Salida

```
2
0
2
1
```

Límites

$$0 \leq N, M \leq 10^6$$

$$1 \leq k \leq 10^7$$

Solución

Una forma poco eficiente de resolver este problema sería, para cada pregunta, recorrer toda la lista y contar cuántas veces aparece el número k . Si hay M preguntas, eso haría que repitiéramos el mismo trabajo muchas veces, dando una complejidad de $O(NM)$, lo cual puede ser demasiado lento.

La idea eficiente es usar un arreglo de ocurrencias. Como los valores de los elementos están en el rango de 1 a 10^7 , podemos crear un arreglo `ocurr` de tamaño $10^7 + 1$, donde en la posición `ocurr[x]` guardemos cuántas veces aparece el número x en la lista. Un arreglo de

este tamaño, usando enteros de 4 bytes, ocupa aproximadamente 40 MB, así que todavía es manejable en memoria en muchos casos.

Entonces, en lugar de guardar la lista completa para contestar las preguntas después, mientras leemos los N elementos vamos aumentando su contador correspondiente. Por ejemplo, si leemos un 3, hacemos `ocurr[3]++`. Eso significa que el número 3 ha aparecido una vez más en la lista. Después, para cada pregunta con valor k , la respuesta ya está calculada: simplemente imprimimos `ocurr[k]`.

De esta manera, construir el arreglo de ocurrencias cuesta $O(N)$, responder las M preguntas cuesta $O(M)$, y la complejidad total es $O(N + M)$.

Código

El código se muestra a continuación*:

```
1. #include <iostream>
2. using namespace std;
3. int occur[100000000 + 1] = {};
4.
5. int main() {
6.     int n;
7.     cin >> n;
8.     for(int i = 0; i < n; ++i) {
9.         int p;
10.        cin >> p;
11.        occur[p] += 1;
12.    }
13.    int m;
14.    cin >> m;
15.    for(int i = 0; i < m; ++i) {
16.        int k;
17.        cin >> k;
18.        cout << occur[k] << "\n";
19.    }
20. }
```

*Nota: Si N y M son muy grandes es necesario **desincronizar** `iostream` y `stdio` (aquí no se hizo para no hacer tan largo el programa). Además, si el rango de k es muy grande, el arreglo debe de estar declarado **fuera** del `main`.

Sumas de prefijos en una lista

Te dan una lista de N elementos, y después te dan M preguntas, cada una formada por dos enteros l y r , que representan un intervalo de la lista. Para cada pregunta, debes saber cuál es la suma de los elementos del intervalo desde la posición l hasta la posición r .

Entrada

Un entero N , seguido de N enteros que forman la lista. Después un entero M , seguido de M pares de enteros l y r , que son los intervalos de las preguntas.

Salida

Para cada pregunta, imprimir un entero que sea la suma del intervalo l y r .

Ejemplo

Entrada:

```
5
2 4 1 3 5
4
1 3
2 5
3 3
4 5
```

Salida:

```
7
13
1
8
```

Límites

$$1 \leq N, M \leq 10^6$$

Solución

Una forma poco eficiente de resolver este problema sería, para cada pregunta, recorrer el intervalo desde la posición l hasta la posición r y sumar sus elementos. Si hacemos eso para las M preguntas, en el peor caso la complejidad puede llegar a ser $O(NM)$, lo cual puede ser demasiado lento.

La idea eficiente es usar un arreglo de sumas de prefijos. Construimos un arreglo **pref** donde **pref[i]** guarda la suma de los primeros i elementos de la lista. Es decir, **pref[0] = 0**, **pref[1]** guarda el primer elemento, **pref[2]** la suma de los dos primeros, y así sucesivamente. Con este arreglo, la suma de los elementos entre las posiciones l y r se puede obtener restando:

$$\text{pref}[r] - \text{pref}[l - 1]$$

Esto funciona porque **pref[r]** contiene la suma desde la posición 1 hasta la r , y al restarle **pref[l - 1]** eliminamos lo que había antes del intervalo que nos interesa.

De esta manera, construir el arreglo de prefijos cuesta $O(N)$, responder las M preguntas cuesta $O(M)$, y la complejidad total es $O(N + M)$.

Código

El código se muestra a continuación*:

```
1. #include <iostream>
2. using namespace std;
3.
4. int main() {
5.     int n;
6.     cin >> n;
7.     long long pref[1000000 + 1];
8.     for(int i = 1; i <= n; ++i) {
9.         int x;
10.        cin >> x;
11.        pref[i] = pref[i - 1] + x;
12.    }
13.    int m;
14.    cin >> m;
15.    for(int i = 0; i < m; ++i) {
```

```

16.         int l, r;
17.         cin >> l >> r;
18.         cout << pref[r] - pref[l - 1] << "\n";
19.     }
20. }

```

*Nota: Aquí se usa `long long` en el arreglo de prefijos porque la suma de muchos elementos puede ser grande, aunque cada elemento individual no lo sea.

Problema de la mayor suma de un subarreglo de tamaño k (ventana deslizante)

Te dan una lista de N enteros y un entero k . Debes encontrar la mayor suma posible de un subarreglo de tamaño exactamente k .

Entrada

Un entero N , seguido de N enteros que forman la lista. Después, un entero k .

Salida

Imprimir un entero que indique la mayor suma de un subarreglo continuo de tamaño k .

Ejemplo

Entrada

```

5
1 2 1 3 2
3

```

Salida

```

6

```

Límites

$$1 \leq k \leq N \leq 10^6$$

Solución

Una forma poco eficiente de resolver este problema sería revisar todos los subarreglos continuos de tamaño k y calcular su suma desde cero. Como hay $N - k + 1$ subarreglos de ese tamaño, y cada uno tarda $O(k)$ en sumarse, la complejidad total sería $O(N \cdot k)$, lo cual puede ser demasiado lento.

La idea eficiente es usar una ventana deslizante de tamaño fijo. Primero calculamos la suma de los primeros k elementos. Esa será la suma de la primera ventana. Después, para mover la ventana una posición a la derecha, ya no hace falta volver a sumar todo. Basta con restar el elemento que sale por la izquierda y sumar el nuevo elemento que entra por la derecha. Así obtenemos la suma de la nueva ventana en tiempo $O(1)$. Repitiendo este proceso para todas las ventanas, podemos ir guardando la mayor suma encontrada. De esta manera, solo recorreremos la lista una vez, y la complejidad total es $O(N)$.

Código

El código se muestra a continuación:

```

1. #include <iostream>
2. using namespace std;
3.
4. int main() {
5.     int n;

```

```

6.     cin >> n;
7.     int arr[1000000 + 1];
8.     for(int i = 0; i < n; ++i) {
9.         cin >> arr[i];
10.    }
11.    int k;
12.    cin >> k;
13.    long long suma = 0;
14.    for(int i = 0; i < k; ++i) {
15.        suma += arr[i];
16.    }
17.    long long mejor = suma;
18.    for(int i = k; i < n; ++i) {
19.        suma = suma + arr[i] - arr[i - k];
20.        if(suma > mejor) {
21.            mejor = suma;
22.        }
23.    }
24.    cout << mejor << "\n";
25. }

```

Apuntadores, la biblioteca `algorithm` y estructuras

Apuntadores

Un apuntador es una variable que guarda la dirección de memoria de otra variable. La sintaxis para declarar un apuntador es **tipo* nombre**, donde tipo es el tipo de dato al que apunta y nombre es el nombre del apuntador.

```
string* p;    //apuntador a cadena
```

Para guardar la dirección de memoria de una variable en un apuntador, se usa el operador `&`:

```
int x = 5;
int* p = &x;    //p guarda la dirección de memoria de x
```

Para obtener el valor al que apunta un apuntador, usamos el operador `*`:

```
cout << *p << "\n";    //imprime 5
*p = 10;                //ahora x vale 10
cout << x << "\n";    //imprime 10
```

Si un apuntador apunta a un elemento de un arreglo, podemos movernos en el arreglo sumando o restando posiciones:

```
int arr[5] = {2, 4, 5, 7, 9}
int* p = &arr[0];    //apunta al primer elemento del arreglo
cout << *p << "\n";    //imprime 2
p++;                //movemos el apuntador una posición del arreglo
cout << *p << "\n";    //imprime 4
p += 3;            //movemos el apuntador tres posiciones del arreglo
cout << *p << "\n";    //imprime 9
```

También podemos restar apuntadores si ambos apuntan a elementos del mismo arreglo, el resultado es cuántas posiciones hay entre ellos:

```
int arr[5] = {1, 3, 5, 7, 8}
int* a = &arr[1];
int* b = &arr[4];
cout << b - a << "\n"; // imprime 3
```

Algunas funciones básicas de la biblioteca algorithm

A continuación, se muestra una tabla con las funciones básicas que tiene la biblioteca **algorithm** de C++.

Función	¿Para qué sirve?	Cómo se usa	Qué regresa
max	Mayor de dos valores	max(a, b)	el valor mayor
min	Menor de dos valores	min(a, b)	el valor menor
swap	Intercambia dos valores	swap(a, b)	no regresa nada
reverse	Invierte un arreglo o intervalo	reverse(&arr[0], &arr[0] + n)	no regresa nada
max_element	Mayor elemento de un arreglo	max_element(&arr[0], &arr[0] + n)	un apuntador al mayor
min_element	Menor elemento de un arreglo	min_element(&arr[0], &arr[0] + n)	un apuntador al menor
count	Cuenta ocurrencias	count(&arr[0], &arr[0] + n, x)	cuántas veces aparece
find	Busca un valor	find(&arr[0], &arr[0] + n, x)	apuntador a la posición

La resta de apuntadores se usa mucho para obtener índices:

```
int arr[5] = {4, 8, 1, 9, 3}
int* p = max_element(&arr[0], &arr[0] + 5);
int pos = p - &arr[0]; //guarda el índice de la posición a la que apunta p en el arreglo
cout << pos << "\n"; //imprime 3
```

Declaración de estructuras

En C++, una estructura es muy parecida a una clase, pero sus atributos y funciones son públicos de manera predeterminada, un ejemplo de una estructura que guarda los datos de un alumno puede ser la siguiente:

```
struct alumno {
    string nombre;
    int edad;
    int matricula;
};
```

Podemos declarar un “objeto” de la estructura de la siguiente manera:

```
alumno ejemplo = {"Pedro", 12, 22222222} //Se debe de declarar en el mismo orden
```

Para acceder a un atributo de la estructura, se hace de la siguiente manera:

```
cout << ejemplo.nombre << "\n"; //Imprime "Pedro"
```

Si queremos acceder a un atributo de una estructura, de la cual tenemos guardada su dirección de memoria en un apuntador, se puede hacer de dos formas: desapuntando el apuntador entre paréntesis, o usando el operador “flechita”:

```
alumno* p = &ejemplo;
cout << (*p).nombre << "\n";    //Imprime Pedro
cout << p->nombre << "\n";      //Imprime Pedro también
```

Algoritmos de ordenamiento y de búsqueda de secuencias

Ordenamiento de secuencias

Para ordenar elementos en C++, normalmente usamos la función **sort**, que se encuentra en la biblioteca **algorithm**.

Esta función recibe dos apuntadores o iteradores, que indican el inicio y el final del intervalo que queremos ordenar (nosotros vamos a considerar el final del intervalo como la posición siguiente al último elemento del intervalo).

Por ejemplo, si queremos ordenar un arreglo **arr** de tamaño n , escribimos:

```
sort(&arr[0], &arr[0] + n);
```

Aquí **&arr[0]** apunta al primer elemento del arreglo, y **&arr[0] + n** apunta a la posición siguiente al último elemento, es decir al final del arreglo.

De manera predeterminada, **sort** ordena de menor a mayor:

```
int arr[5] = {4, 1, 3, 5, 2};
sort(&arr[0], &arr[0] + 5);
```

Después de esto, el arreglo queda así:

```
1 2 3 4 5
```

La complejidad de **sort** es, en general, $O(n \log(n))$, por lo que es muy eficiente para arreglos grandes.

También es posible indicar una forma distinta de ordenar usando un predicado. Un predicado es una función que recibe dos elementos y decide cuál debe de ir primero. El predicado debe de regresar **true** si el primer elemento debe de ir antes que el segundo, y **false** en caso contrario. Por ejemplo, si queremos ordenar una lista de enteros de mayor a menor, podemos hacer lo siguiente:

```
bool mayor(int& a, int& b) {
    return a > b;
}
```

Esta función, en general, se debe de declarar fuera del **main**, y nosotros la mandamos a llamar en la misma función de **sort**, de la siguiente manera:

```
sort(&arr[0], &arr[0] + n, mayor);
```

También Podemos decidir cómo ordenar listas de estructuras u objetos usando predicados. Por ejemplo, para la estructura alumno de la sección anterior, si queremos ordenar un arreglo de alumnos, primero por edad, y en caso de empate, alfabéticamente por nombre, podemos declarar el siguiente predicado:

```
bool comp(alumno& a, alumno& b) {
    if(a.edad != b.edad) {
        return a.edad < b.edad;
    }
    return a.nombre < b.nombre;
}
```

Aquí, primero revisamos si las edades son distintas. Si lo son, ponemos primero al alumno con menor edad. Si las edades son iguales, entonces comparamos los nombres en orden alfabético.

Búsqueda eficiente sobre secuencias ordenadas

La búsqueda binaria se usa para buscar elementos en un **arreglo ordenado** de manera muy eficiente. Su complejidad es de $O(\log(n))$ para un arreglo de tamaño n .

La función más básica para esto en C++ es **binary_search**, que se encuentra en la biblioteca **algorithm**. Por ejemplo, si queremos saber si existe el valor x en un arreglo de n elementos ordenados, lo podemos hacer de la siguiente manera:

```
bool res = binary_search(&arr[0], &arr[0] + n, x);
```

es decir, le pasamos a la función el inicio y el final del arreglo de donde queremos buscar, y el valor x . La función regresa **true** en caso de que si exista ese elemento en el arreglo y **false** en caso contrario.

Además de **binary_search**, en programación competitiva se usan mucho **lower_bound** y **upper_bound**. Estas funciones no solo dicen si un elemento existe, sino que regresan un apuntador a cierta posición del arreglo.

Función	¿Para qué sirve?	Cómo se usa	Qué regresa
binary_search	Verifica si un valor existe en un arreglo ordenado	binary_search(&arr[0], &arr[0] + n, x)	true o false
lower_bound	Busca la primera posición mayor o igual que x	lower_bound(&arr[0], &arr[0] + n, x)	apuntador a la primera posición $\geq x$
upper_bound	Busca la primera posición estrictamente mayor que x	upper_bound(&arr[0], &arr[0] + n, x)	apuntador a la primera posición $> x$

Un ejemplo de del uso de **lower_bound** y **upper_bound** puede ser el siguiente:

```
int arr[5] = {1, 2, 5, 7, 8};
int* res_lb = lower_bound(&arr[0], &arr[0] + 5, 7);
int* res_ub = upper_bound(&arr[0], &arr[0] + 5, 7);
cout << res_lb << " " << res_ub << "\n";    // Imprime: 7 8
```

Por último, podemos usar estas mismas funciones para buscar otros tipos de datos, como estructuras, usando el mismo predicado con el que está ordenado el arreglo.

Si buscamos si existe el **alumno** con nombre “Pedro” y edad 12 en una lista ordenada con el predicado **comp** de la sección anterior, lo podemos hacer de la siguiente manera:

```
alumno arr[n];
...
alumno x = {"Pedro", 12, 22222222};
bool res = binary_search(&arr[0], &arr[0] + n, x, comp); //Ponemos comp como parámetro
cout << res << "\n"; //Imprime 1 o 0
```

Estructuras de datos lineales

Cadenas de caracteres (string)

Una **string** es una estructura de la biblioteca estándar de C++ que se usa para guardar y manipular texto. La sintaxis general para declarar una cadena es `string nombre`. Por ejemplo:

```
string cad;
string nombre = "Pedro";
```

También podemos leer una cadena con `cin`:

```
string cad;
cin >> cad;
```

Esto lee una palabra, es decir, **se detiene en el primer espacio**.

Si queremos leer una **línea completa**, usamos `getline`:

```
string cad;
getline(cin, cad);
```

Para acceder a los caracteres de una cadena usamos corchetes y podemos imprimir con **cout**:

```
string cad = "hola";
cout << cad[0] << "\n"; //imprime h
cout << cad[2] << "\n"; //imprime l
cout << cad << "\n"; //imprime hola
```

También podemos agregar caracteres o texto al final:

```
string cad = "ho";
cad.push_back('l'); //ahora cad vale "hol"
cad += "a como estas"; //ahora cad vale "hola como estas"
```

Las cadenas también se pueden comparar alfabéticamente:

```
string a = "ana", "luis";
if(a < b) {
    cout << "ana va antes que luis\n";
}
```


A continuación, se presenta una tabla con las funciones básicas de string:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
size	Dice cuántos caracteres tiene	cad.size()	la cantidad de caracteres
empty	Verifica si la cadena está vacía	cad.empty()	true o false
push_back	Agrega un carácter al final	cad.push_back(c)	agrega c al final
pop_back	Quita el último carácter	cad.pop_back()	no regresa nada
substr	Obtiene una subcadena	cad.substr(i, len)	la subcadena desde i de longitud len
find	Busca una subcadena o carácter	cad.find("abc")	la posición o string::npos
clear	Borra todo el contenido	cad.clear()	no regresa nada
front	Da el primer carácter	cad.front()	el primer carácter
back	Da el último carácter	cad.back()	el último carácter

Arreglos dinámicos (vector)

Un **vector** es una estructura de la biblioteca estándar de C++ que funciona como un arreglo dinámico. A diferencia de un arreglo normal, su tamaño puede cambiar durante la ejecución del programa.

La sintaxis general para declarar un vector es **vector<tipo> nombre**, por ejemplo:

```
vector<int> v;           //vector de enteros
vector<string> nombres; //vector de cadenas
```

También podemos declarar un vector con un tamaño inicial:

```
vector<int> v(5); //vector de 5 enteros
```

Si queremos declarar un vector, con un tamaño inicial, y con todos sus elementos con cierto valor, lo hacemos de la siguiente forma:

```
vector<int> v(5, -1) //vector de 5 enteros inicializados con -1
```

Para acceder a un elemento, usamos corchetes igual que en los arreglos:

```
cout << v[0] << "\n";
```

A continuación, se presenta una tabla con las funciones básicas de vector

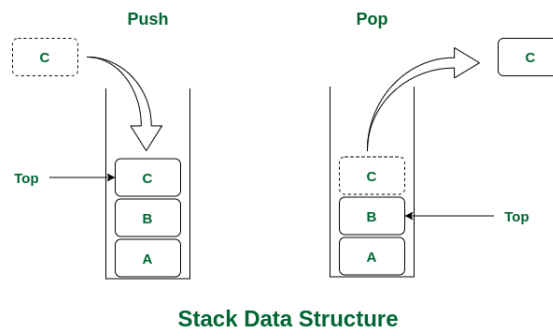
Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push_back	Agrega un elemento al final	v.push_back(x)	agrega x al final
pop_back	Quita el último elemento	v.pop_back()	no regresa nada
size	Dice cuántos elementos tiene el vector	v.size()	la cantidad de elementos
empty	Verifica si el vector está vacío	v.empty()	true o false
back	Da el último elemento	v.back()	el último elemento
front	Da el primer elemento	v.front()	el primer elemento

Si queremos un apuntador al inicio o al final del vector, podemos usar las funciones `begin()` y `end()`. Por ejemplo, si queremos ordenar un vector:

```
sort(v.begin(), v.end());
```

Pilas (stack)

Una **stack** es una estructura de la biblioteca estándar de C++ que sigue la idea de último en entrar, primero en salir. Es decir, el último elemento que se mete a la pila es el primero que se puede sacar.



La sintaxis general para declarar una pila es **`stack<tipo> nombre`**. Por ejemplo:

```
stack<int> pila;
stack<string> nombres;
```

A continuación, se presenta una tabla con las funciones básicas de **stack**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push	Mete un elemento a la pila	<code>pila.push(x)</code>	agrega x arriba
pop	Quita el elemento de arriba	<code>pila.pop()</code>	no regresa nada
top	Da el elemento de arriba	<code>pila.top()</code>	el elemento superior
size	Dice cuántos elementos tiene la pila	<code>pila.size()</code>	la cantidad de elementos
empty	Verifica si la pila está vacía	<code>pila.empty()</code>	true o false

Una observación importante es que **`pop()` no regresa** el elemento que quita. Si quieres guardar ese valor, primero debes usar **`top()`** y después **`pop()`**.

```
int x = pila.top();
pila.pop();
```

Colas (queue)

Una **queue** es una estructura de la biblioteca estándar de C++ que sigue la idea de primero en entrar, primero en salir. Es decir, el primer elemento que entra a la cola es el primero que sale.



Queue Data Structure

La sintaxis general para declarar una cola es **queue<tipo> nombre**. Por ejemplo:

```
queue<int> cola;
queue<string> nombres;
```

A continuación, se presenta una tabla con las funciones básicas de **queue**:

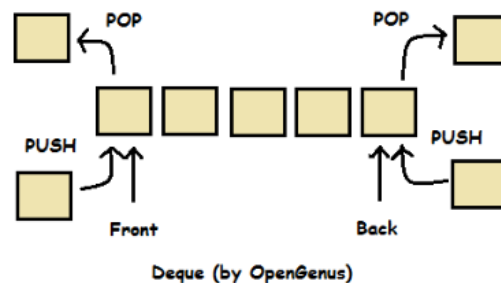
Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push	Mete un elemento a la cola	cola.push(x)	agrega x al final
pop	Quita el elemento del frente	cola.pop()	no regresa nada
front	Da el elemento del frente	cola.front()	el primer elemento
back	Da el elemento del final	cola.back()	el último elemento
size	Dice cuántos elementos tiene la cola	cola.size()	la cantidad de elementos
empty	Verifica si la cola está vacía	cola.empty()	true o false

Al igual que en **stack**, **pop()** **no regresa** el elemento que quita. Si quieres guardarlo, primero debes usar **front()** y después **pop()**.

```
int x = cola.front();
cola.pop();
```

Dobles colas (deque)

Un **deque** es una estructura de la biblioteca estándar de C++ parecida a un vector, pero con la ventaja de que permite agregar y quitar elementos tanto al inicio como al final de manera eficiente. La palabra deque viene de *double ended queue*, es decir, una cola con dos extremos.



La sintaxis general para declarar un deque es **deque<tipo> nombre**. Por ejemplo:

```
deque<int> d;  
deque<string> nombres;
```

También podemos acceder a cualquier posición usando corchetes, igual que en un arreglo o en un vector:

```
cout << d[1] << "\n";
```

A continuación, se presenta una tabla con las funciones básicas de **deque**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push_back	Agrega un elemento al final	d.push_back(x)	agrega x al final
push_front	Agrega un elemento al inicio	d.push_front(x)	agrega x al inicio
pop_back	Quita el último elemento	d.pop_back()	no regresa nada
pop_front	Quita el primer elemento	d.pop_front()	no regresa nada
front	Da el primer elemento	d.front()	el primer elemento
back	Da el último elemento	d.back()	el último elemento
size	Dice cuántos elementos tiene	d.size()	la cantidad de elementos
empty	Verifica si está vacío	d.empty()	true o false

Listas enlazadas (list)

Una **list** es una estructura de la biblioteca estándar de C++ que permite agregar y quitar elementos de manera eficiente, tanto al inicio como al final, y también insertar o borrar elementos en posiciones intermedias si ya tenemos un iterador a esa posición.

La sintaxis general para declarar una lista es **list<tipo> nombre**. Por ejemplo:

```
list<int> lista;  
list<string> nombres;
```

Como una **list** no se recorre con índices, normalmente se usa un iterador:

```
for(list<int>::iterator it = lista.begin(); it != lista.end(); ++it) {  
    cout << *it << " ";  
}
```

Si tenemos un iterador a una posición de la lista, podemos agregar o eliminar elementos en esa posición de la siguiente manera:

```
list<int> lista;  
lista.push_back(10); lista.push_back(20);  
list<int>::iterator it = lista.begin(); ++it;  
lista.insert(it, 15);  
lista.erase(it);
```

A continuación, se presenta una tabla con las funciones básicas de **list**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push_back	Agrega un elemento al final	lista.push_back(x)	agrega x al final
push_front	Agrega un elemento al inicio	lista.push_front(x)	agrega x al inicio
pop_back	Quita el último elemento	lista.pop_back()	no regresa nada
pop_front	Quita el primer elemento	lista.pop_front()	no regresa nada
front	Da el primer elemento	lista.front()	el primer elemento
back	Da el último elemento	lista.back()	el último elemento
insert	Inserta un elemento antes de una posición	lista.insert(it, x)	inserta x antes de it (un iterador)
erase	Borra el elemento en una posición	lista.erase(it)	borra el elemento apuntado por it (un iterador)
size	Dice cuántos elementos tiene	lista.size()	la cantidad de elementos
empty	Verifica si está vacía	lista.empty()	true o false
begin	Da un iterador al inicio	lista.begin()	iterador al primer elemento
end	Da un iterador al final lógico	lista.end()	iterador a la posición siguiente al último

Estructuras de datos no lineales

Conjuntos (set)

Un set es una estructura de la biblioteca estándar de C++ que guarda elementos **ordenados** y **sin repetir**.

La sintaxis general para declarar un set es **set<tipo> nombre**. Por ejemplo:

```
set<int> s;
set<string> nombres;
```

Para agregar elementos, usamos **insert**:

```
s.insert(10);
s.insert(5);
s.insert(20);
s.insert(10);
```

Aunque se intente insertar 10 dos veces, en el set solo aparecerá una vez. Para recorrer un set, usamos iteradores:

```
for(set<int>::iterator it = s.begin(); it != s.end(); ++it) {
    cout << *it << " ";
}
```

Para eliminar elementos, si existen, usamos **erase**:

```
s.erase(5);
```

A continuación, se presenta una tabla con las funciones básicas de `set`:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>insert</code>	Agrega un elemento	<code>s.insert(x)</code>	inserta x si no estaba
<code>erase</code>	Borra un elemento	<code>s.erase(x)</code>	borra x si existe
<code>count</code>	Verifica si un elemento existe	<code>s.count(x)</code>	1 o 0
<code>find</code>	Busca un elemento	<code>s.find(x)</code>	iterador al elemento o <code>s.end()</code>
<code>size</code>	Dice cuántos elementos tiene	<code>s.size()</code>	la cantidad de elementos
<code>empty</code>	Verifica si está vacío	<code>s.empty()</code>	true o false
<code>begin</code>	Da un iterador al inicio	<code>s.begin()</code>	iterador al menor elemento
<code>end</code>	Da un iterador al final lógico	<code>s.end()</code>	iterador a la posición siguiente al último

Diccionarios (*map*)

Un `map` es una estructura que guarda pares de la forma **clave – valor**. Las claves están ordenadas y no se repiten.

La sintaxis general es `map<tipo_clave, tipo_valor> nombre`. Por ejemplo:

```
map<string, int> edad;
```

Aquí la clave es un `string` y el valor es un `int`. Podemos guardar valores así:

```
edad["Ana"] = 18;
edad["Luis"] = 20;
edad["Carlos"] = 19;
```

Podemos acceder al valor asociado a una clave de la siguiente manera:

```
string nombre = "Ana";
cout << edad[nombre] << "\n";
```

Podemos recorrer un `map` usando iteradores. Aquí, `(*it).first` es la clave y `(*it).second` es el valor:

```
for(map<string, int>::iterator it = edad.begin(); it != edad.end(); ++it) {
    cout << (*it).first << " " << (*it).second << "\n";
}
```

A continuación, se presenta una tabla con las funciones básicas de `map`:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
<code>insert</code>	Inserta un par clave - valor	<code>m.insert({a, b})</code>	inserta el par {clave, valor} si la clave no estaba
<code>erase</code>	Borra una clave	<code>m.erase(x)</code>	borra la clave x si existe
<code>count</code>	Verifica si una clave existe	<code>m.count(x)</code>	1 o 0
<code>find</code>	Busca una clave	<code>m.find(x)</code>	iterador a la clave o <code>m.end()</code>
<code>size</code>	Dice cuántos pares tiene	<code>m.size()</code>	la cantidad de pares

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
empty	Verifica si está vacío	m.empty()	true o false
begin	Da un iterador al inicio	m.begin()	iterador al primer par
end	Da un iterador al final lógico	m.end()	iterador a la posición siguiente al último

Colas de prioridad (priority_queue)

Una **priority_queue** es una estructura parecida a una cola, pero en lugar de sacar el primero que entró, saca primero el elemento de **mayor prioridad**.

En C++, por defecto, una **priority_queue** saca primero el elemento **más grande**. Se encuentra en la biblioteca **queue**.

La sintaxis general es **priority_queue<tipo> nombre**. Por ejemplo:

```
priority_queue<int> pq;
```

A continuación, se muestra una lista de las funciones básicas de **priority_queue**:

Función	¿Para qué sirve?	Cómo se usa	Qué hace o qué regresa
push	Mete un elemento	pq.push(x)	agrega x a la estructura
pop	Quita el elemento de mayor prioridad	pq.pop()	no regresa nada
top	Da el elemento de mayor prioridad	pq.top()	el elemento superior
size	Dice cuántos elementos tiene	pq.size()	la cantidad de elementos
empty	Verifica si está vacía	pq.empty()	true o false

Si queremos usar una **priority_queue** con una estructura creada por nosotros, o simplemente queremos ordenar de otra forma, C++ necesita saber cómo comparar dos elementos para decidir cuál tiene mayor prioridad. Para eso, podemos **sobrecargar el operador <** dentro de la estructura, o fuera del **main**, usando la misma idea que los predicados de ordenamiento:

```
struct alumno {
    string nombre;
    int edad;
    int matricula;
};
bool operator<(const alumno& otro) const {
    if(edad != otro.edad) {
        return edad < otro.edad;
    }
    return nombre > otro.nombre;
}
```

Aquí, el alumno con mayor edad sigue teniendo mayor prioridad. Si dos alumnos tienen la misma edad, quedará arriba el que tenga el nombre alfabéticamente menor. Ya teniendo sobrecargado el operador, nosotros inicializamos la cola de prioridad de la misma manera.

```
priority_queue<alumno> pq;
pq.push({"Pedro", 20});
```

Algunos algoritmos aritméticos básicos

Operaciones con módulo

En programación competitiva es común que el resultado de un problema sea muy grande. Para evitar desbordamientos, a veces el enunciado pide imprimir la respuesta módulo algún número, por ejemplo $10^9 + 7$.

Si tenemos dos números a y b , y un módulo m , las operaciones básicas se pueden hacer así:

$$\begin{aligned}(a + b) \% m &= ((a \% m) + (b \% m)) \% m \\(a - b) \% m &= ((a \% m) - (b \% m) + m) \% m \\(a \cdot b) \% m &= ((a \% m) \cdot (b \% m)) \% m\end{aligned}$$

La idea es que podemos ir tomando módulo en cada paso para que los números no crezcan demasiado.

Por ejemplo, en lugar de hacer:

```
ans = ans + x;
```

podemos hacer:

```
ans = (ans + x) % mod
```

También es común guardar primero el módulo de cada número:

```
a %= mod;
b %= mod;
```

y luego operar:

```
suma = (a + b) % mod;
producto = (a * b) % mod;
```

Cuando hay muchas operaciones, lo importante es no esperar hasta el final si el número puede crecer demasiado. Conviene tomar módulo después de cada suma o multiplicación importante.

Por ejemplo:

```
1. uint64_t mod = 1000000007;
2. uint64_t ans = 0;
3. for(int i = 0; i < n; ++i) {
4.     ans = (ans + arr[i]) % mod;
5. }
```

La división módulo no se maneja igual que una división normal, por eso normalmente se estudia aparte.

Criba de Eratóstenes

Sirve para calcular qué números de 0 a n son primos. El arreglo `es_primo[i]` queda en `true` si i es primo. Complejidad: $O(n \log(\log(n)))$.

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. vector<bool> criba(uint64_t n) {
5.     vector<bool> es_primo(n + 1, true);
6.
7.     if(n >= 0) es_primo[0] = false;
8.     if(n >= 1) es_primo[1] = false;
9.
10.    for(uint64_t i = 2; i <= n / i; ++i) {
11.        if(es_primo[i]) {
12.            for(uint64_t j = i * i; j <= n; j += i) {
13.                es_primo[j] = false;
14.            }
15.        }
16.    }
17.
18.    return es_primo;
19. }
```

Cómo usarlo:

```
1. uint64_t n = 100;
2. vector<bool> es_primo = criba(n);
3. if(es_primo[37]) {
4.     cout << "37 es primo\n";
5. }
```

Coeficiente binomial iterativo

Calcula el coeficiente binomial $\binom{n}{k}$ de manera muy eficiente. Es útil cuando el resultado todavía cabe en `uint64_t`. Complejidad: $O(k)$, usando $k = \min(k, n - k)$.

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. uint64_t binomial(uint64_t n, uint64_t k) {
5.     if(k > n) {
6.         return 0;
7.     }
8.     if(k > n - k) {
9.         k = n - k;
10.    }
11.    uint64_t resultado = 1;
12.    for(uint64_t i = 1; i <= k; ++i) {
13.        resultado = resultado * (n - i + 1);
14.        resultado = resultado / i;
15.    }
16.    return resultado;
17. }
```

Cómo usarlo:

```
1. uint64_t r = binomial(5, 2);
2. cout << r << "\n"; // imprime 10
```

Potencia rápida con módulo

Calcula $a^b \% m$ de manera eficiente. Complejidad: $O(\log(b))$.

```
1. #include < bits/stdc++.h >
2. using namespace std;
3.
4. uint64_t potencia_mod(uint64_t base, uint64_t exponente, uint64_t mod) {
5.     uint64_t resultado = 1;
6.     base %= mod;
7.     while(exponente > 0) {
8.         if(exponente % 2 == 1) {
9.             resultado = (resultado * base) % mod;
10.        }
11.        base = (base * base) % mod;
12.        exponente /= 2;
13.    }
14.    return resultado;
15. }
```

Cómo usarlo:

```
1. uint64_t MOD = 1000000007;
2. uint64_t r = potencia_mod(2, 10, MOD);
3. cout << r << "\n"; // imprime 1024
```

Binomial iterativo con módulo

Calcula $\binom{n}{k} \% m$ usando inversos modulares. Esta versión funciona cuando m es primo. Complejidad: $O(k \log(m))$, usando $k = \min(k, m - k)$.

```
1. #include < bits/stdc++.h >
2. using namespace std;
3.
4. uint64_t potencia_mod(uint64_t base, uint64_t exponente, uint64_t mod) {
5.     uint64_t resultado = 1;
6.     base %= mod;
7.     while(exponente > 0) {
8.         if(exponente % 2 == 1) {
9.             resultado = (resultado * base) % mod;
10.        }
11.        base = (base * base) % mod;
12.        exponente /= 2;
13.    }
14.    return resultado;
15. }
16.
17. uint64_t inverso_mod(uint64_t x, uint64_t mod) {
18.     return potencia_mod(x, mod - 2, mod);
19. }
20.
21. uint64_t binomial_mod(uint64_t n, uint64_t k, uint64_t mod) {
22.     if(k > n) {
23.         return 0;
24.     }
25. }
```

```

24.     }
25.     if(k > n - k) {
26.         k = n - k;
27.     }
28.     uint64_t resultado = 1;
29.     for(uint64_t i = 1; i <= k; ++i) {
30.         resultado = (resultado * ((n - i + 1) % mod)) % mod;
31.         resultado = (resultado * inverso_mod(i, mod)) % mod;
32.     }
33.     return resultado;
34. }

```

Cómo usarlo:

```

1. uint64_t MOD = 1000000007;
2. uint64_t r = binomial_mod(5, 2, MOD);
3. cout << r << "\n"; // imprime 10

```

Factorización básica

Descompone un número n como producto de factores primos. Regresa parejas de la forma {primo, exponente}. Complejidad: $O(\sqrt{n})$.

```

1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. vector<pair<uint64_t, uint64_t>> factorizar(uint64_t n) {
5.     vector<pair<uint64_t, uint64_t>> factores;
6.     for(uint64_t p = 2; p <= n / p; ++p) {
7.         if(n % p == 0) {
8.             uint64_t exponente = 0;
9.             while(n % p == 0) {
10.                 n /= p;
11.                 exponente++;
12.             }
13.             factores.push_back({p, exponente});
14.         }
15.     }
16.
17.     if(n > 1) {
18.         factores.push_back({n, 1});
19.     }
20.     return factores;
21. }

```

Cómo usarlo, calculando los factores de $n = 360$:

```

1. uint64_t n = 360;
2. vector<pair<uint64_t, uint64_t>> factores = factorizar(n);
3. for(int i = 0; i < factores.size(); ++i) {
4.     cout << factores[i].first << " " << factores[i].second << "\n";
5. }

```

Salida, las parejas {primo, exponente} de la factorización $360 = 2^3 \cdot 3^2 \cdot 5^1$:

```

2 3
3 2

```

Máximo común divisor y mínimo común múltiplo

En la biblioteca numeric desde C++17 están disponibles las funciones de máximo común divisor (gcd) y mínimo común múltiplo (lcm)

```
1. uint64_t a = 24;
2. uint64_t b = 36;
3. uint64_t mcd = gcd(a, b);
4. uint64_t mcm = lcm(a, b);
```

Algunas consideraciones extras

Uso de auto

En C++, la palabra auto permite que el compilador deduzca automáticamente el tipo de una variable. Esto es muy útil cuando el tipo de dato es **muy largo, difícil de escribir** o simplemente **incómodo de leer**.

Por ejemplo, en lugar de escribir:

```
vector<int>::iterator it = v.begin();
```

podemos escribir:

```
auto it = v.begin();
```

y el compilador deduce automáticamente el tipo correcto.

Esto se usa mucho en programación competitiva cuando trabajamos con:

- iteradores
- **set**
- **map**
- **vector**
- funciones de la biblioteca estándar que regresan tipos largos

Por ejemplo:

```
set<int> s;
auto it = s.find(7);
```

Aquí, **s.find(7)** regresa un iterador de **set<int>**, pero no hace falta escribir todo el tipo.

También es común en **map**:

```
map<string, int> m;
auto it = m.begin();
```

o en recorridos:

```
for(auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

y también:

```
for(auto it = m.begin(); it != m.end(); ++it) {  
    cout << (*it).first << " " << (*it).second << "\n";  
}
```

La ventaja principal de **auto** es que hace el código **más corto** y muchas veces **más claro**, sobre todo cuando el tipo completo no aporta mucho y solo estorba visualmente.

¿Qué hacer cuando voy a resolver un problema?

Del prólogo de las notas del Club de Algoritmos UAM-Azcapotzalco, escritas por el Dr. Rodrigo Alexander Castro Campos, considera lo siguiente al resolver un problema:

Antes de enviar:

- Escribe algunos casos de prueba simples si los ejemplos no son suficientes.
- ¿Están cerca los límites de tiempo? Si es así, genera casos máximos.
- ¿El uso de memoria está dentro de los límites?
- ¿Podría haber algún desbordamiento?
- Asegúrate de enviar el archivo correcto.

Respuesta incorrecta:

- ¿Imprime tu solución! Imprime también la salida de depuración.
- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?
- ¿Puede tu algoritmo manejar todo el rango de entrada?
- Lee nuevamente el enunciado completo del problema.
- ¿Manejas todos los casos límite correctamente?
- ¿Has entendido correctamente el problema?
- ¿Hay variables no inicializadas?
- ¿Algún desbordamiento?
- ¿Confundes N y M, i y j, etc.?
- ¿Estás seguro de que tu algoritmo funciona?
- ¿Qué casos especiales no has considerado?
- ¿Estás seguro de que las funciones STL que usas funcionan como piensas?
- Añade algunas aserciones, tal vez vuelva a enviar.
- Crea algunos casos de prueba para probar tu algoritmo.
- Revisa el algoritmo con un caso simple.
- Revisa esta lista nuevamente.

- Explica tu algoritmo a un compañero de equipo.
- Pídele a un compañero que revise tu código.
- Da un pequeño paseo, por ejemplo, al baño.
- ¿Es correcto tu formato de salida? (incluyendo espacios en blanco)
- Reescribe tu solución desde el principio o deja que un compañero lo haga.

Error de tiempo de ejecución:

- ¿Has probado todos los casos límite localmente?
- ¿Hay variables no inicializadas?
- ¿Estás leyendo o escribiendo fuera del rango de algún vector?
- ¿Alguna aserción que podría fallar?
- ¿Alguna posible división por 0? (por ejemplo, mod 0)
- ¿Alguna posible recursión infinita?
- ¿Punteros o iteradores inválidos?
- ¿Estás usando demasiada memoria?
- Depura con reenvíos (por ejemplo, señales remapeadas, ver Varios).

Límite de tiempo excedido:

- ¿Tienes algún bucle infinito posible?
- ¿Cuál es la complejidad de tu algoritmo?
- ¿Estás copiando muchos datos innecesarios? (usa referencias)
- ¿Qué tan grandes son la entrada y salida? (considera scanf)
- Evita vector, map. (usa arrays o unordered_map)
- ¿Qué piensan tus compañeros sobre tu algoritmo?

Límite de memoria excedido:

- ¿Cuál es la cantidad máxima de memoria que debería necesitar tu algoritmo?

Fuente: <https://github.com/darkmx/uam-icpc/>

- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?